

✓ Step 16 — One-Hot Encoding for Categorical Features

Why One-Hot Encoding?

Previously, we used `LabelEncoder` to convert categorical text (like 'Brand' or 'Engine Type') into numerical values. While simple, `LabelEncoder` assigns arbitrary integer values (e.g., Audi=0, BMW=1, Toyota=2). For linear models, this implies an ordinal relationship ($0 < 1 < 2$) that might not exist in reality.

One-Hot Encoding creates new binary columns for each category. For example, instead of a single 'Brand' column, we get 'Brand_Audi', 'Brand_BMW', 'Brand_Toyota', etc. A car will have a **1** in its corresponding brand column and **0** in others.

This approach ensures that the model treats each category as independent, which is more appropriate for linear models.

💡 We'll re-do the encoding and then proceed with tuning Lasso.

```
# First, let's reload the original DataFrame to revert the Label Encoding
# and then drop the 'Model' column and apply log transformation to 'Price' again
```

```
# This ensures we start with a clean slate for One-Hot Encoding.

# Note: This block assumes the 'df' variable from initial loading is still available or can be reloaded.
# If the notebook is run top-down, 'df' would have been modified. For robustness, let's reload the data.

# Reload the dataset from scratch
import kagglehub
from kagglehub import KaggleDatasetAdapter

file_path = "1.04. Real-life example.csv"
df_ohe = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "smritisingh1997/car-sales.csv",
    file_path
)

# Handle missing values as before
df_ohe = df_ohe.dropna(subset=['Price', 'EngineV'])

# Remove outliers from EngineV (<= 10 litres)
df_ohe = df_ohe[df_ohe['EngineV'] <= 10]

# Apply Log Transformation
df_ohe['Log_Price'] = np.log(df_ohe['Price'])

# Drop original 'Price' and 'Model' column
df_ohe = df_ohe.drop(columns=['Model', 'Price'], axis=1)

# Identify categorical columns for One-Hot Encoding
cat_cols_ohe = df_ohe.select_dtypes(include=['object']).columns

# Apply One-Hot Encoding
df_ohe = pd.get_dummies(df_ohe, columns=cat_cols_ohe, drop_first=True)

display(df_ohe.head())
```

```
/tmp/ipykernel_2108/1693219979.py:13: DeprecationWarning: Use dataset_load() instead of load_dataset(). load_
df_ohe = kagglehub.load_dataset(
Using Colab cache for faster access to the 'car-salescsv' dataset.
```

	Mileage	EngineV	Year	Log_Price	Brand_BMW	Brand_Mercedes-Benz	Brand_Mitsubishi	Brand_Renault	Brand_Toyota
0	277	2.0	1991	8.342840	True	False	False	False	False
1	427	2.9	1999	8.974618	False	True	False	False	False
2	358	5.0	2003	9.495519	False	True	False	False	False
3	240	4.2	2007	10.043249	False	False	False	False	False
4	120	2.0	2011	9.814656	False	False	False	False	True

✓ Step 17 — Re-split Data with One-Hot Encoded Features

Now that our categorical features are One-Hot Encoded, we need to re-split our dataset into training and testing sets. This ensures that our Lasso model will be trained and evaluated on the new feature representation.

```
# Define features (X) and target (y) with the new df_ohe
X_ohe = df_ohe.drop(columns=['Log_Price'])
y_ohe = df_ohe['Log_Price']

# Split the data into training and testing sets
# Using the same random_state for reproducibility
X_train_ohe, X_test_ohe, y_train_ohe, y_test_ohe = train_test_split(
    X_ohe, y_ohe, test_size=0.2, random_state=42
)

print("Shape of X_train_ohe:", X_train_ohe.shape)
print("Shape of X_test_ohe:", X_test_ohe.shape)
```

```
Shape of X_train_ohe: (3204, 18)
Shape of X_test_ohe: (801, 18)
```

✓ Step 18 — Tune Lasso `alpha` with Cross-Validation

The `alpha` parameter in Lasso Regression controls the strength of the regularization. A larger `alpha` increases the penalty for non-zero coefficients, leading to more features being shrunk to zero. Choosing the right `alpha` is crucial for optimal model performance.

We will use `LassoCV`, which performs cross-validation to find the best `alpha` within a specified range. This automates the tuning process and helps prevent overfitting.

```
from sklearn.linear_model import LassoCV

# Define a range of alpha values to test
# eps is the ratio of the smallest alpha to the largest alpha in the grid
# n_alphas is the number of alphas along the regularization path
# cv is the number of folds for cross-validation

lasso_cv_model = LassoCV(alphas=None, cv=5, max_iter=10000, random_state=42, n_jobs=-1)
lasso_cv_model.fit(X_train_ohe, y_train_ohe)

optimal_alpha = lasso_cv_model.alpha_
print(f"Optimal alpha found by LassoCV: {optimal_alpha}")

# Train the final Lasso model with the optimal alpha
lasso_tuned_model = Lasso(alpha=optimal_alpha, max_iter=10000, random_state=42)
lasso_tuned_model.fit(X_train_ohe, y_train_ohe)
```

Optimal alpha found by LassoCV: 0.057521675431919495

```
▼ Lasso ⓘ ?
Lasso(alpha=np.float64(0.057521675431919495), max_iter=10000, random_state=42)
```

✓ Step 19 — Evaluate Tuned Lasso Model

Now, let's evaluate the performance of our Lasso model trained with the optimal `alpha` and One-Hot Encoded features.

```
# Predictions on the test set
lasso_tuned_y_predict = lasso_tuned_model.predict(X_test_ohe)

# Calculate R² and RMSE
lasso_tuned_r2 = r2_score(y_test_ohe, lasso_tuned_y_predict)
lasso_tuned_mse = mean_squared_error(y_test_ohe, lasso_tuned_y_predict)
lasso_tuned_rmse = np.sqrt(lasso_tuned_mse)

print(f"Tuned Lasso Test R²: {lasso_tuned_r2}")
print(f"Tuned Lasso Test RMSE: {lasso_tuned_rmse}")

# Compare with previous Lasso results
models_results["Lasso_Tuned_OHE"] = {"R2": lasso_tuned_r2, "RMSE": lasso_tuned_rmse}
results_df = pd.DataFrame(models_results).T
display(results_df)
```

Tuned Lasso Test R²: 0.7503128255403587
Tuned Lasso Test RMSE: 0.4699738504599247

	R2	RMSE	
Linear	0.829630	0.388215	
Ridge	0.829622	0.388224	
Lasso	0.752040	0.468346	
Lasso_Tuned_OHE	0.750313	0.469974	

Next steps: [Generate code with results_df](#) [New interactive sheet](#)

✓ Step 20 — Analyze Tuned Lasso Coefficients (with One-Hot Encoding)

With One-Hot Encoding, the coefficients for categorical features are now directly interpretable. Let's examine which features the tuned Lasso model retained and their importance.

```
# Get feature names from the one-hot encoded training data
feature_names_ohc = X_train_ohc.columns

# Create a DataFrame for Lasso coefficients
lasso_ohc_coef_df = pd.DataFrame({
    'Feature': feature_names_ohc,
    'Lasso_Coeff_Tuned': lasso_tuned_model.coef_
})

# Calculate absolute coefficients for ranking
lasso_ohc_coef_df['Lasso_Abs_Tuned'] = lasso_ohc_coef_df['Lasso_Coeff_Tuned'].abs()

# Sort by absolute coefficient to see most important features
display(lasso_ohc_coef_df.sort_values('Lasso_Abs_Tuned', ascending=False).head(15))

# Features removed by tuned Lasso (coefficients exactly zero)
print("\nFeatures removed by Tuned Lasso:")
display(lasso_ohc_coef_df[lasso_ohc_coef_df['Lasso_Coeff_Tuned'] == 0])
```

	Feature	Lasso_Coeff_Tuned	Lasso_Abs_Tuned
1	EngineV	0.331396	0.331396
2	Year	0.085255	0.085255
17	Registration_yes	0.062350	0.062350
0	Mileage	-0.001575	0.001575
4	Brand_Mercedes-Benz	0.000000	0.000000
3	Brand_BMW	0.000000	0.000000
6	Brand_Renault	-0.000000	0.000000
7	Brand_Toyota	0.000000	0.000000
8	Brand_Volkswagen	-0.000000	0.000000
5	Brand_Mitsubishi	-0.000000	0.000000
9	Body_hatch	-0.000000	0.000000
10	Body_other	0.000000	0.000000
12	Body_vagon	-0.000000	0.000000
11	Body_sedan	-0.000000	0.000000
13	Body_van	-0.000000	0.000000

Features removed by Tuned Lasso:

	Feature	Lasso_Coeff_Tuned	Lasso_Abs_Tuned
3	Brand_BMW	0.0	0.0
4	Brand_Mercedes-Benz	0.0	0.0
5	Brand_Mitsubishi	-0.0	0.0
6	Brand_Renault	-0.0	0.0
7	Brand_Toyota	0.0	0.0
8	Brand_Volkswagen	-0.0	0.0
9	Body_hatch	-0.0	0.0
10	Body_other	0.0	0.0
11	Body_sedan	-0.0	0.0
12	Body_vagon	-0.0	0.0
13	Body_van	-0.0	0.0
14	Engine Type_Gas	-0.0	0.0
15	Engine Type_Other	-0.0	0.0
16	Engine Type_Petrol	-0.0	0.0

Interpretation of Tuned Lasso with One-Hot Encoding

- **Feature Selection:** Observe which features have a `Lasso_Coeff_Tuned` of exactly `0`. These features were deemed unimportant by Lasso and effectively removed from the model.
- **Direct Interpretability:** For One-Hot Encoded features (e.g., `Brand_BMW`, `Engine Type_Diesel`), their non-zero coefficients now indicate the specific impact of that category compared to the base (dropped) category.
- **Effect of Alpha:** The optimal `alpha` found by `LassoCV` strikes a balance between model complexity and performance. It regularizes enough to perform feature selection but not so much that it underfits the data.

This refined Lasso model provides a more robust understanding of feature importance, especially for categorical variables, and potentially a more parsimonious model.

Automatic Feature Selection: The most significant advantage of Lasso (and the reason it's often preferred for feature selection) is its ability to shrink some coefficients exactly to zero. This effectively removes those features from the model, leading to a simpler and more interpretable model. Linear Regression and Ridge Regression, on the other hand, only shrink coefficients; they don't eliminate features entirely. In the notebook, we saw that Lasso set the coefficients for 'EngineV', 'Engine Type', and 'Registration' to zero, effectively removing them from the model, unlike Linear and Ridge models.

Model Simplicity and Interpretability: By driving unimportant feature coefficients to zero, Lasso produces a more sparse model. A model with fewer features is easier to understand, explain, and potentially faster to run in production.

Reduced Overfitting: When you have many features, some of which might be noise or highly correlated, removing them can help prevent the model from overfitting the training data, leading to better generalization to new, unseen data.

Step 21 — Tune Ridge `alpha` with Cross-Validation

Similar to Lasso, Ridge Regression also has a regularization parameter, `alpha`, which controls the strength of the penalty. We use `RidgeCV` to automatically find the best `alpha` for our Ridge model through cross-validation. This ensures that our Ridge model is well-tuned to our data, especially after applying One-Hot Encoding.

```
from sklearn.linear_model import RidgeCV, Ridge

# Define a range of alpha values to test. RidgeCV can automatically determine a good range.
# cv is the number of folds for cross-validation

ridge_cv_model = RidgeCV(alphas=np.logspace(-6, 6, 13), cv=5)
ridge_cv_model.fit(X_train_ohe, y_train_ohe)

optimal_ridge_alpha = ridge_cv_model.alpha_
print(f"Optimal alpha found by RidgeCV: {optimal_ridge_alpha}")

# Train the final Ridge model with the optimal alpha
ridge_tuned_model = Ridge(alpha=optimal_ridge_alpha, random_state=42)
ridge_tuned_model.fit(X_train_ohe, y_train_ohe)
```

Optimal alpha found by RidgeCV: 0.1

▼ Ridge ⓘ ?
Ridge(alpha=np.float64(0.1), random_state=42)

Step 22 — Evaluate Tuned Ridge Model

Let's assess the performance of our Ridge model after tuning its `alpha` parameter with cross-validation. We will calculate the R^2 score and RMSE to understand how well it generalizes to unseen data, and then compare it with the other models.

```
# Predictions on the test set
ridge_tuned_y_predict = ridge_tuned_model.predict(X_test_ohe)

# Calculate R² and RMSE
ridge_tuned_r2 = r2_score(y_test_ohe, ridge_tuned_y_predict)
ridge_tuned_mse = mean_squared_error(y_test_ohe, ridge_tuned_y_predict)
ridge_tuned_rmse = np.sqrt(ridge_tuned_mse)

print(f"Tuned Ridge Test R²: {ridge_tuned_r2}")
print(f"Tuned Ridge Test RMSE: {ridge_tuned_rmse}")

# Compare with previous models
models_results["Ridge_Tuned_OHE"] = {"R2": ridge_tuned_r2, "RMSE": ridge_tuned_rmse}
results_df = pd.DataFrame(models_results).T
display(results_df)
```

Tuned Ridge Test R^2 : 0.8711248961841458
Tuned Ridge Test RMSE: 0.3376448197045691

	R2	RMSE	
Linear	0.829630	0.388215	
Ridge	0.829622	0.388224	
Lasso	0.752040	0.468346	
Lasso_Tuned_OHE	0.750313	0.469974	
Ridge_Tuned_OHE	0.871125	0.337645	

Next steps: [Generate code with results_df](#) [New interactive sheet](#)

It appears that tuning the Ridge model with One-Hot Encoding has yielded the best performance in terms of both R^2 (closest to 1) and RMSE (lowest error). This model explains a higher percentage of the variance in car prices and has a lower average prediction

Step 23 — Interpret Tuned Ridge Model Coefficients

We will now examine the coefficients of the tuned Ridge model. This helps us understand the importance and direction of influence for each feature on car prices. Ridge regression's characteristic is to shrink these coefficients without completely eliminating any features, making all features contribute to the prediction.

```
[229]
# Get feature names from the one-hot encoded training data
feature_names_ohc = X_train_ohc.columns

# Create a DataFrame for Ridge coefficients
ridge_ohc_coef_df = pd.DataFrame({
    'Feature': feature_names_ohc,
    'Ridge_Coeff_Tuned': ridge_tuned_model.coef_
})

# Calculate absolute coefficients for ranking
ridge_ohc_coef_df['Ridge_Abs_Tuned'] = ridge_ohc_coef_df['Ridge_Coeff_Tuned'].abs()

# Sort by absolute coefficient to see most important features
display(ridge_ohc_coef_df.sort_values('Ridge_Abs_Tuned', ascending=False).head(15))

print("\nRidge Coefficients (sorted by absolute value, top 15):")
```

	Feature	Ridge_Coeff_Tuned	Ridge_Abs_Tuned
17	Registration_yes	0.904509	0.904509
6	Brand_Renault	-0.632034	0.632034
5	Brand_Mitsubishi	-0.480824	0.480824
13	Body_van	-0.443145	0.443145
9	Body_hatch	-0.383010	0.383010
12	Body_vagon	-0.334175	0.334175
11	Body_sedan	-0.304736	0.304736
10	Body_other	-0.250510	0.250510
1	EngineV	0.239295	0.239295
8	Brand_Volkswagen	-0.236409	0.236409
7	Brand_Toyota	-0.220895	0.220895
14	Engine Type_Gas	-0.140226	0.140226
15	Engine Type_Other	-0.120711	0.120711
16	Engine Type_Petrol	-0.119218	0.119218
2	Year	0.080198	0.080198

Ridge Coefficients (sorted by absolute value, top 15):

Interpretation of Tuned Ridge Coefficients

- No Zero Coefficients:** As expected with Ridge regression, none of the coefficients are exactly zero. This confirms that Ridge shrinks the magnitude of coefficients but retains all features in the model.
- Magnitude of Coefficients:** Features with larger absolute coefficients (e.g., `EngineV`, `Year`, `Mileage`) are considered more influential in predicting the `Log_Price`.
- Direction of Influence:** The sign of the coefficient indicates the direction of the relationship. For instance, a positive coefficient for `EngineV` means larger engine volume is associated with higher car prices (after log transformation).

Ridge regression is effective at handling multicollinearity (when features are highly correlated) by distributing the impact across correlated features rather than selecting just one, as Lasso might. It aims for a more stable model by preventing any single feature from dominating the prediction due to large coefficient values.

Top Influential Features: 'Registration_yes', 'Brand_Renault', 'Brand_Mitsubishi', and 'Body_van' appear to have the largest absolute coefficients, indicating a strong influence on car price. 'EngineV' and 'Year' also show significant coefficients.

No Zero Coefficients: As we discussed earlier, and as confirmed by these results, Ridge Regression shrinks coefficients but does not set them exactly to zero. This means all features, including the One-Hot Encoded categorical variables, are retained in the model, even if their individual contribution is very small.

Direction of Influence: The sign of the coefficient still indicates the direction of the relationship. For example, a positive coefficient for 'Registration_yes' (0.9045) suggests that cars with registration tend to have higher prices (compared to the baseline or Registration_no, which was dropped during One-Hot Encoding). Conversely, negative coefficients for 'Brand_Renault' (-0.632) or 'Body_van' (-0.443) suggest these categories are associated with lower prices compared to their respective base categories.